Advanced Rust 2026 - Lab 5: Unsafe Rust Exercises - Teacher Copy

Lukáš Hozda

Spring 2026

Exercise 1: Manually Allocated Fixed Vector

Questions and Answers

1. Why must `Drop` only touch initialized elements?

Dropping a value runs `T`'s destructor. An uninitialized slot is not a `T`. Calling `drop_in_place` on it would let arbitrary bit patterns be interpreted as a valid value of type `T`, which is undefined behavior.

1. Why must `pop` lower `len` before or immediately after reading the element?

After `ptr.read()`, the slot no longer contains an initialized `T` owned by the vector. If `len` still includes that slot, later `Drop` or slice creation may treat moved-out memory as initialized. In this implementation, `len` is lowered before the read and there is no user code between the two operations.

1. Which methods create references from raw pointers, and what must be true at

that point?

`get`, `get_mut`, `as_slice`, and `as_mut_slice` create references. The pointer must be non-null, aligned, inside the allocation, and point only at initialized elements. The aliasing rules must also hold: `get_mut` and `as_mut_slice` require exclusive access, which is provided by `&mut self`.

1. What extra complications appear for zero-sized types?

Zero-sized types do not need allocation, but a collection can still logically contain many of them. Pointer arithmetic on zero-sized types does not advance by bytes, capacity arithmetic is special, and deallocating a zero-sized layout is not the same case as deallocating ordinary storage. The student exercise permits rejecting zero-sized types to keep the invariants local.

Reference Solution

```rust
use std::alloc::{self, Layout};
use std::mem;
use std::ptr::NonNull;

pub struct RawFixedVec<T> {
    ptr: NonNull<T>,
    len: usize,
    cap: usize,
}

impl<T> RawFixedVec<T> {
    pub fn with_capacity(capacity: usize) -> Self {
        assert!(
            mem::size_of::<T>() != 0,
            "zero-sized types are not supported by this exercise implementation"
        );

        if capacity == 0 {
            return Self {
                ptr: NonNull::dangling(),
                len: 0,
                cap: 0,
            };
        }

        let layout = Layout::array::<T>(capacity).unwrap();
```

```
        let raw = unsafe { alloc::alloc(layout) };
        let ptr = NonNull::new(raw.cast::<T>())
            .unwrap_or_else(|| alloc::handle_alloc_error(layout));

        Self { ptr, len: 0, cap: capacity }
    }

    pub fn capacity(&self) -> usize {
        self.cap
    }

    pub fn len(&self) -> usize {
        self.len
    }

    pub fn push(&mut self, value: T) -> Result<(), T> {
        if self.len == self.cap {
            return Err(value);
        }

        unsafe {
            self.ptr.as_ptr().add(self.len).write(value);
        }
        self.len += 1;
        Ok(())
    }

    pub fn pop(&mut self) -> Option<T> {
        if self.len == 0 {
            return None;
        }

        self.len -= 1;
        Some(unsafe { self.ptr.as_ptr().add(self.len).read() })
    }

    pub fn get(&self, index: usize) -> Option<&T> {
        if index >= self.len {
            return None;
        }

        unsafe { Some(&*self.ptr.as_ptr().add(index)) }
    }

    pub fn get_mut(&mut self, index: usize) -> Option<&mut T> {
        if index >= self.len {
            return None;
        }

        unsafe { Some(&mut *self.ptr.as_ptr().add(index)) }
    }

    pub fn as_slice(&self) -> &[T] {
        unsafe { std::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
    }

    pub fn as_mut_slice(&mut self) -> &mut [T] {
        unsafe { std::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
    }
}

impl<T> Drop for RawFixedVec<T> {
    fn drop(&mut self) {
        while self.len > 0 {
            self.len -= 1;
```

```
            unsafe {
                std::ptr::drop_in_place(self.ptr.as_ptr().add(self.len));
            }
        }

        if self.cap != 0 {
            let layout = Layout::array::<T>(self.cap).unwrap();
            unsafe {
                alloc::dealloc(self.ptr.cast::<u8>().as_ptr(), layout);
            }
        }
    }
}
```

Important tests:

```
#[test]
fn pushes_until_full() {
    let mut vec = RawFixedVec::with_capacity(2);
    assert_eq!(vec.push(10), Ok(()));
    assert_eq!(vec.push(20), Ok(()));
    assert_eq!(vec.push(30), Err(30));
    assert_eq!(vec.as_slice(), &[10, 20]);
}

#[test]
fn pops_in_reverse_order() {
    let mut vec = RawFixedVec::with_capacity(3);
    vec.push(1).unwrap();
    vec.push(2).unwrap();
    vec.push(3).unwrap();
    assert_eq!(vec.pop(), Some(3));
    assert_eq!(vec.pop(), Some(2));
    assert_eq!(vec.pop(), Some(1));
    assert_eq!(vec.pop(), None);
}
```

Exercise 2: Packed Header Parser

Questions and Answers

1. Why is `read_unaligned` used instead of taking `&u16` or `&u32`?

The input is a byte slice, so the starting address has alignment 1. The u16/u32 fields appear at protocol offsets, not necessarily at addresses aligned for `u16` or `u32`. Creating `&u16` or `&u32` would assert alignment. A raw pointer plus `read_unaligned` performs the load without forming an invalid reference.

1. Why would transmuting the flags byte to `bool` be wrong?

`bool` has only two valid values, 0 and 1. A flags byte is a bitset and can legitimately contain values such as 2 or 3. The correct operation is a bit test, not reinterpretation of the whole byte as a Rust `bool`.

1. Which checks establish that the returned payload slice is valid?

`parse_header` checks that the header bytes exist before reading fields. `split_packet` uses `checked_add` for `HEADER_LEN + payload_len` and checks `packet.len() ≥ payload_end` before slicing. The returned payload slice is a subslice of the original packet and cannot outlive it.

Reference Solution

```
pub const HEADER_LEN: usize = 10;

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct Header {
    pub version: u8,
    pub compressed: bool,
    pub encrypted: bool,
    pub payload_len: u16,
    pub checksum: u32,
```

```
}

pub fn parse_header(bytes: &[u8]) -> Option<Header> {
    if bytes.len() < HEADER_LEN {
        return None;
    }
    if &bytes[0..2] != b"RS" {
        return None;
    }

    let version = bytes[2];
    if version == 0 {
        return None;
    }

    let flags = bytes[3];
    if flags & !0b0000_0011 != 0 {
        return None;
    }

    Some(Header {
        version,
        compressed: flags & 0b0000_0001 != 0,
        encrypted: flags & 0b0000_0010 != 0,
        payload_len: read_u16_le(bytes, 4),
        checksum: read_u32_le(bytes, 6),
    })
}

pub fn split_packet(packet: &[u8]) -> Option<(Header, &[u8])> {
    let header = parse_header(packet)?;
    let payload_end = HEADER_LEN.checked_add(header.payload_len as usize)?;
    if packet.len() < payload_end {
        return None;
    }
    Some((header, &packet[HEADER_LEN..payload_end]))
}

fn read_u16_le(bytes: &[u8], offset: usize) -> u16 {
    debug_assert!(offset + 2 <= bytes.len());
    let ptr = unsafe { bytes.as_ptr().add(offset).cast::<u16>() };
    unsafe { u16::from_le(std::ptr::read_unaligned(ptr)) }
}

fn read_u32_le(bytes: &[u8], offset: usize) -> u32 {
    debug_assert!(offset + 4 <= bytes.len());
    let ptr = unsafe { bytes.as_ptr().add(offset).cast::<u32>() };
    unsafe { u32::from_le(std::ptr::read_unaligned(ptr)) }
}
```

Test data:

```
fn valid_packet() -> [u8; 15] {
    [
        b'R', b'S',
        3,
        0b0000_0011,
        5, 0,
        0x44, 0x33, 0x22, 0x11,
        b'h', b'e', b'l', b'l', b'o',
    ]
}
```

Exercise 3: Miri Failures

Expected Results

`cargo test` can pass for all four tests. It may produce a warning for the `transmute`, but warnings are not runtime UB detection.

`cargo +nightly miri test decodes_entry` should report construction of an invalid `bool` at the `transmute::<u8, bool>` call.

`cargo +nightly miri test serializes_record` should report uninitialized memory in the byte array created by transmuting a `repr(C)` struct with padding.

`cargo +nightly miri test updates_counter` should report a Stacked Borrows violation. The raw pointer is created first, then a fresh `&mut` borrow invalidates it, and the raw pointer is used afterwards.

`cargo +nightly miri test reads_packed_len` should report an alignment violation. The field pointer has alignment 1 because it points into a packed struct, but `ptr.read()` requires alignment suitable for `u16`.

Sound replacements:

1. Decode boolean flags with bit tests or comparisons.
2. Serialize records field-by-field into an initialized byte buffer.
3. Do not keep using stale raw pointers across fresh references to the same place.
4. Use `std::ptr::read_unaligned` for packed or protocol-offset integer reads.

Broken Code From Student Copy

```rust
#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct Entry {
    pub tag: u8,
    pub enabled: bool,
}

pub fn decode_entry(bytes: [u8; 2]) -> Entry {
    Entry {
        tag: bytes[0],
        enabled: unsafe { std::mem::transmute::<u8, bool>(bytes[1]) },
    }
}

#[repr(C)]
#[derive(Clone, Copy)]
pub struct DiskRecord {
    pub tag: u8,
    pub counter: u32,
}

pub fn record_bytes(record: DiskRecord) -> [u8; std::mem::size_of::<DiskRecord>()] {
    unsafe { std::mem::transmute(record) }
}

pub fn increment_twice() -> i32 {
    let mut value = 40_i32;
    let raw = &mut value as *mut i32;
    let reference = &mut value;
    *reference += 1;
    unsafe {
        *raw += 1;
    }
    value
}

#[repr(C, packed)]
pub struct PackedHeader {
    pub tag: u8,
    pub len: u16,
}

pub fn packed_len() -> u16 {
```

```
    let header = PackedHeader { tag: 1, len: 0x1234_u16.to_le() };
    let ptr = std::ptr::addr_of!(header.len);
    unsafe { u16::from_le(ptr.read()) }
}
```